

Contents

OpenSpec AI Development Harness 面试八股文	1
一、电梯演讲（三档长度，先背熟）	1
二、背景：解决什么问题（为什么做）	2
三、核心概念（名词解释，被问到就按这个答）	2
四、架构与技术选型	3
4.1 分层	3
4.2 技术选型及理由（必考）	3
4.3 「四个世界」知识分离	4
4.4 项目结构逐层讲解（面试常问「具体怎么组织的」）	4
五、关键设计决策（面试核心，每条都有「为什么」）	6
5.1 状态推进单入口（Transition Service）	6
5.2 双 Gate 分工	6
5.3 AI 执行 / 框架裁决（Skill 设计）	6
5.4 Prompt 体系职责边界	6
5.5 目录体系：纯业务名 + 锚点（有演进故事，见六-1）	7
5.6 product/features = SSOT 派生缓存（方案 D，最新重构）	7
5.7 多仓交付（DU fan-out / fan-in）	7
5.8 断点续跑	7
5.9 幂等与自检	7
5.10 产品规则人工评审晋升制（product/specs）	8
六、难点与解决故事（每个 1 分钟，讲出「问题 → 方案 → 代价」）	8
6-1 目录改名漂移（体现演进能力）	8
6-2 防止 Agent 绕过流程（体现安全设计）	8
6-3 多仓一致性（体现分布式思维）	8
6-4 文档即数据库的一致性（体现细节）	8
七、工程质量（数字 + 做法）	8
八、高频追问 Q&A（尖锐问题预演）	9
九、数字卡片（考前最后过一遍）	10
十、开场白模板（串场用）	10

OpenSpec AI Development Harness 面试八股文

用途：面试讲解项目用。按「电梯演讲 → 背景 → 概念 → 架构 → 设计决策 → 难点故事 → 质量 → 追问 Q&A → 数字卡片」组织，可直接背。数据以 2026-09 时点为准（379 测试 / Phase 3.8）。

一、电梯演讲（三档长度，先背熟）

30 秒版：

我做的是一个面向 AI Coding Agent 的工程控制层，叫 OpenSpec AI Development Harness。它解决的问题是：现在 AI 编码工具（Trae、Cursor、Claude Code）写代码很快，但没有项目记忆、没有流程约束、交付不可追溯。我的方案是把「规范驱动开发（SDD）」做成一个 CLI 框架：Agent 每做一个需求，都要走 explore→PRD→设计→任务→开发→测试→评审→收敛 的完整链路，每个阶段有机器门禁和人工门禁，产物落盘留痕，知

识回流到项目知识库。它不调模型、不替代 Agent，只负责告诉 Agent「做什么、读什么、守什么规则、产出什么」。

1 分钟版（在 30 秒版基础上补）：

技术上它是 Node.js 原生 ESM + 零构建的 CLI，依赖只有 4 个（commander、@clack/prompts、picocolors、yaml）。核心设计有三点：第一，**双世界分离**——知识世界（产品树、标准、规范）和交付世界（Change 变更单元）物理分开，代码仓不掺知识文件；第二，**状态机收口**——所有 Change 状态推进必须走唯一的 Transition Service，Agent 没有任何直改状态的口子，配合 artifact hash 校验防止「改了产物还用过期门禁结果」；第三，**AI 执行、框架裁决**——11 个 SDD Skill 只写方法论让 Agent 执行，裁决权在确定性的 Machine Gate 和人负责的 Human Gate。多仓场景用 Delivery Unit 做 fan-out/fan-in，断点续跑靠 Workflow Engine 的 WAITING 状态机。测试 379 个全绿。

3 分钟版：在 1 分钟版基础上，把下面「二、背景」和「六、难点」各讲 1 个，收尾讲数字。

二、背景：解决什么问题（为什么做）

传统 AI 编程的四个痛点（面试开场用）：

- 1. **没有项目记忆**：每次会话都是白纸，AI 不知道产品有哪些模块、哪些能力已交付。
- 2. **没有工程约束**：AI 直接从需求跳到代码，跳过设计评审、测试证据。
- 3. **不可追溯**：为什么这么改？依据哪条需求？谁批准的？没有任何留痕。
- 4. **知识不沉淀**：改完代码，学到的经验（技术选型、踩坑）没有回流。

OpenSpec 的解法是把 AI 编码从「需求 → AI → 代码」改成闭环：

需求 → 知识 → 规范 → 实现 → 证据 → 知识更新

一句话定位：**AI Coding Agent 的工程控制层（Harness），不是模型平台、不是 Agent Runtime、不是 IDE。**

三、核心概念（名词解释，被问到就按这个答）

概念	一句话解释
Workspace	项目根目录，.sdd/ 标记，独立 Git 仓，不含业务代码
Feature Tree	四级特性树：Product → Module(L1) → Feature(L2) → Capability(L3) → Story(L4)，feature-tree.yaml 是唯一权威源（SSOT）
Change (CHG)	一次需求交付的完整单元，delivery/changes/CHG-XXXX/，有 9 态生命周期
feature-path	Change 绑定到树上的一条四级链（L1/L2/L3/Story），决定了产物落盘目录
DU (Delivery Unit)	多仓交付单元：1 DU = 1 个子仓库的可执行交付规格，task 阶段 fan-out，dev/test 回传 fan-in
Machine Gate	确定性机检（产物存在、checkbox 声明项、hash 一致），只检查明确声明为机检的条目

概念	一句话解释
Human Gate	人审（业务正确性、范围合理性），approve ≠ 通过生命周期，只登记状态
Transition Service	唯一合法的状态推入口，推进前验证 artifact hash 与 Gate 记录一致
Workflow Engine	不做 AI 推理，负责技能调度、产物就绪检测、Gate 评估、状态推进、断点续跑
SDD Skill	11 个 Agent 可执行的技能 (sdd-explore/prd/design/task/dev/test/review/converge/reverse/feature-tree/knowledge), yaml 元数据 + SKILL.md 方法论 + checklist + gate 定义
Evidence 审计包	证据系统：dev/test 阶段强制留下可验证证据（测试结果、commit、截图） 归档后的 Change 目录：全产物 + 证据 + 完整骨架，只读永不改

Change 生命周期 (9 态, 背下来):

created → exploring → specified → designed → tasked
→ developing → testing → completed → archived

Workflow 断点状态 (5 个返回值):

ADVANCED / WAITING_FOR_ARTIFACT / WAITING_FOR_MACHINE_FIX / WAITING_FOR_HUMAN / COMPLETED

四、架构与技术选型

4.1 分层

cli/openspec/	← 命令行交互层 (commander + @clack/prompts) , 薄壳
core/	← 领域内核, 全部导出纯函数/纯 IO 函数
workspace/	← init/upgrade/sync/ide 等工作区工程
sdd/	← SDD 领域 (change/gate/feature/du/skill/workflow 7 大族)
skills/	← 11 个 Skill 资产 (yaml + SKILL.md + checklist + gate)
templates/	← init 时复制到 Workspace 的脚手架
prompts/	← 5 族 prompt (persona 按 Skill 粒度)

4.2 技术选型及理由 (必考)

选择	理由 (面试口头版)
零构建、纯 JS、原生 ESM 仅 4 个依赖	工具类项目，用户是装了就跑的 CLI；没有构建链 = 没有版本漂移、没有编译产物一致性负担
node:fs/promises 原生 API, 不用 fs-extra yaml 的 parseDocument + setIn	commander (命令解析)、@clack/prompts (交互)、picocolors (颜色)、yaml (Document API 保留注释)。依赖越少，供应链风险和升级成本越低 Node 20+ 的 fs/promises 已覆盖递归 cp/rm/mkdir, 没必要多拉依赖 改 metadata 时保留用户注释——文档即接口，注释不能丢

选择	理由（面试口头版）
core 写纯函数 文件系统即数据库	所有领域逻辑输入输出明确、可单测；CLI 层只做参数解析和展示 一切状态落 YAML + Markdown，Git 天然版本化 + diff 可读 + 离线可用；不引入服务端

4.3 「四个世界」知识分离

- **Standards World** (技术规则)、**Product World** (产品树/需求)、**Delivery World** (CHG/DU/证据)、**Implementation World** (各业务子仓代码)。
- Workspace 仓不包含 implementation/ 内容，各子仓独立 Git，Workspace 存 commit 指针 (repositories.yaml)。

4.4 项目结构逐层讲解（面试常问「具体怎么组织的」）

4.4.1 Harness 仓总览（框架自身）

```
OpenSpec-AI-Development-Harness/
├── cli/openspec/          # CLI 交互层 (commander) , 15 个命令文件, 只做解析/展示/退出
│   ├── src/
│   │   ├── commands/    # change/du/workflow/gate/feature/skill/doctor/init/upgrade/i
│   │   └── lib/          # workspace-resolver (向上找 .sdd/)、logger (ok/warn/error 统一
├── core/                  # 领域内核: 全部导出纯函数, 不 import cli
│   ├── sdd/              # SDD 领域 33 个模块 (七大族, 见 4.4.2)
│   └── workspace/        # init/upgrade/skill-sync/ide-commands/copier 等工作区工程
├── skills/                # 11 个 Skill 资产: skill.yaml + SKILL.md + checklist + rules + ga
├── prompts/               # 5 族 prompt (common/explore/design/coding/review) , front-
├── templates/             # openspec init 复制到 Workspace 的脚手架 (metadata.yaml 模板等
├── tests/                 # node:test 原生 spec, 379 用例
├── docs/ + plans/        # 架构文档 + 各 phase 设计档案
```

讲解口径：两句话——①「**CLI 薄壳 + 内核纯函数**：命令层不写业务，业务全在 core，所以 CLI 换壳不动内核、每个 core 模块都能直接单测」；②「**资产与代码分离**：skills/prompts/templates 是随 init 复制、随 upgrade 同步的『内容资产』，框架升级 = 资产同步 + schema 迁移，不动用户数据」。

4.4.2 core/sdd 的七大族（33 个模块的地图，被追问时亮出来）

族	模块	职责一句话
Change族 (11)	change-model (metadata 读写) / change-repository (list/show 只读扫描) / change-state-machine (9 态机) / change-validator / change-skeleton (CHG 内四级骨架物化) / change-archiver (归档) / change-id-generator (发号) / candidate-repository / artifact-path (产物落位) / artifact-writer / artifact-hash	Change 的数据、目录、生命周期全管辖

族	模块	职责一句话
Gate 族 (3)	gate-config-loader (读 gate.yaml) / gate-validator (Machine Gate 机检) / gate-repository (结果持久化进 metadata)	机检的配置、执行、留痕三段式
Feature 族 (4)	feature-model (树读取/统一视图/ID 生成) / feature-writer (树写入) / feature-dirname (目录段清洗 + CHG 侧锚点) / feature-materializer (features/ 派生缓存重建)	特性树 SSOT 及其投影
DU 族 (1)	delivery-unit	DU 注册/物化到子仓/commit 指针回传
编排 族 (7)	workflow-engine (断点续跑) / workflow-loader (default.yaml) / transition-service (唯一状态入口) / context-assembler (阶段上下文) / instruction-builder (指令 + Prompt 片段装配) / skill-registry / skill-loader + prompt-loader	Workflow Engine 的「调度-装配-裁决」三件套
知识 族 (3)	evidence-model (证据) / requirement-model (需求) / knowledge-reverser (sdd-reverse 反向提取)	知识沉淀侧
支撑 族 (3)	doctor-checks (体检集) / fs-walker / git-submodule	基础设施

讲解口径：不是背 33 个文件名，而是说「我按领域把内核拆成七个族——Change 管生命周期，Gate 管裁决，Feature 管产品树，DU 管多仓，编排族把 skill 调度和状态推进收口，剩下是知识和支撑。最关键的约束是**依赖方向**：编排族可以调其他族，其他族绝不反向依赖编排族，Transition Service 是全仓唯一写状态的地方」。

4.4.3 生成的 Workspace 结构 (用户视角，讲「落地后长什么样」)

```

ai-platform/
├── .sdd/                # Workspace 标记 + version.yaml (upgrade 依据)
├── product/             # 产品知识世界
│   ├── feature-tree.yaml # 唯一权威源 SSOT
│   ├── features/         # 派生缓存 (先清后建，仅 Story 级 README)
│   └── standards/        # 技术标准 (converge 阶段回流更新)
├── delivery/            # 交付世界
│   ├── changes/CHG-0002/ # 活跃 Change: metadata.yaml + 四级业务名骨架
│   │   └── 平台基座/用户管理/账户能力/用户登录/ # STORY 目录=全部产物+evidence/+DU/
│   └── archive/CHG-0001/ # 审计包 (整体迁移，只读)
├── skills/ + prompts/   # init 从 Harness 复制，skill sync 升级
├── implementation/     # 各业务子仓 (backend/frontend, 独立 Git, Workspace 忽略)
└── repositories.yaml    # 子仓注册 (kind: git/dir)

```

讲解口径：「openspec init 之后的目录本身就是架构的说明书：product 与 delivery 物理分离、implementation 不进 Workspace 的 Git、每个 CHG 目录四级路径直接暴露它在产品树里的位置——**目录结构即领域模型**，新人不用读代码就能看懂。」

4.4.4 结构相关的加分细节

- **依赖方向图** (可在白板上画): cli → core ← 资产 (skills/prompts/templates); core 内部 编排族 → 领域族 → 纯工具, 无环。
- **为什么 cli 和 core 分开**: 未来加 web 视图/CI runner 时复用同一个 core; 命令层是可替换的「皮」。
- **为什么 assets (skills/prompts) 不放 core**: 它们是被 init/upgrade 分发给用户 Workspace 的内容, 有版本号和同步协议, 和「编译进二进制的逻辑」生命周期不同。
- **文件命名即 API**: core 每个文件 = 一个领域概念, 导出的都是该概念的纯函数; 找逻辑先按概念找文件, 不用全文搜。

五、关键设计决策 (面试核心, 每条都有「为什么」)

5.1 状态推进单入口 (Transition Service)

- **问题**: Agent 太灵活, 如果 skill 能直接改状态, 流程就形同虚设。
- **决策**: 禁止 change status --set 直改、禁止 skill run / gate approve 推进生命周期, 一切走 requestTransition。
- **加分点**: 推进时校验当前 artifact hash 与 Gate 保存的 hash 一致——**门禁通过后你又改了产物? Gate 结果自动失效, 重新过检**。这保证了「审批的东西 = 落盘的东西」。

5.2 双 Gate 分工

- **Machine Gate = Deterministic Validation**: 只查机器能可靠判断的 (产物存在、声明的 checkbox、DU 覆盖率、commit 对齐), 不做 AI 语义评分——机器评分不可靠就是不可靠。
- **Human Gate = 人机无法可靠判断的决策**: 业务正确性、范围合理性。bypass 必须留审计记录, bypassed ≠ approved。
- **面试官追问「为什么不让 AI 打分」**: 答——LLM 评分不稳定、可被提示词操纵; 框架的价值恰恰是把「确定性」和「判断力」分开。

5.3 AI 执行 / 框架裁决 (Skill 设计)

- Skill 不调模型, v0.x 由外部 Agent (Trae/Cursor/Claude Code) 按 SKILL.md 方法论执行, 产出写回 CHG 目录, 再由 workflow 推进。
- Skill 运行必须在 Change Context 内 (有 CHG 才能跑阶段技能)。
- **知识防漂移**: IDE 斜杠命令是「薄入口」, 只引导 Agent 跑 workflow run + 按 SKILL.md 执行, 不把方法论复制进命令文件——方法论只有一份 (skill 资产), 命令从 skill registry 程序化渲染, skill sync 后新技能自动获得新命令。

5.4 Prompt 体系职责边界

- persona/通用约束/输出格式 → prompts/ (5 族); 流程方法论 → 留在 SKILL.md。
- 引用机制: skill.yaml prompts: 字段 + SKILL.md 顶部引用行 + instruction-builder 注入「Prompt 片段」section。

- **为什么**：换 Agent 不换方法论，换语气不换流程；SKILL.md 瘦身只抽「行为规则」通用条目。

5.5 目录体系：纯业务名 + 锚点（有演进故事，见六-1）

- 目录段 = 业务名（平台基座/用户管理/账户能力/用户登录），不用 ID 段——人一眼看懂。
- CHG 内部四级骨架的 README front-matter id 作锚点：树改名后重跑 change skeleton 锚点 rename，防名字漂移。
- 同名冲突检测：不同 id 撞同一个清洗后的名字 → 报错而不是自动加后缀（保持路径可读）。

5.6 product/features = SSOT 派生缓存（方案 D，最新重构）

- feature-tree.yaml 是唯一权威源；product/features/ 只是给人在 IDE 里层层点进浏览的派生缓存。
- materialize 语义 = **先清旧缓存、按 SSOT 全量重建**（支持-dry-run 预览差异）。
- L1/L2/L3 只建空目录（承担导航），**只在 Story 级写 README**：面包屑 + 描述 + Change 历史表（changes+archive 全扫）+ 直达审计包的相对链接。
- **为什么**：锚点 rename/drift 那套增量同步逻辑复杂且易漏；派生缓存「不一致就重建」，正确性靠定义保证，代码砍掉一半。教训：**能重建的就不要同步**。

5.7 多仓交付（DU fan-out / fan-in）

- task 阶段把 Story 拆成每仓一个 DU（含 Implementation Sketch/Pseudocode/Verification 三段前置规格），du materialize 把规格物化到各子仓 implementation/<repo>/delivery/....
- dev/test 回传 baseline/result commit（fan-in），Workspace 状态机据此推进。
- 机检：du-coverage（每仓都有 DU）、du-fan-in-testing/complete、submodule-pointer-aligned。
- 仓库形态 kind: git（校验 HEAD）/ dir（普通目录，跳过 commit 对齐检查）——兼容单仓多模块团队。

5.8 断点续跑

- Workflow Engine 每次调 workflow run 只推进一步，返回五态之一；WAITING_FOR_HUMAN 时人来 gate approve，WAITING_FOR_MACHINE_FIX 时 Agent 修产物，再 run 就从断点继续。
- **为什么**：Agent 会话是易失的，流程状态必须外置到磁盘（metadata.yaml + gate 记录），随时 kill 随时恢复。

5.9 幂等与自检

- 几乎所有写操作幂等：skeleton 重跑 = 同步、materialize 重跑 = 重建、ide 重跑 = 版本比对跳过。
- openspec doctor 一条命令体检：结构/字段/版本/多仓/CHG 骨架锚点一致性/features 缓存健康度。
- openspec upgrade 四步固定（sync → 补文件 → schema 迁移 → 版本记录）+ --rollback 可回退。

5.10 产品规则人工评审晋升制 (product/specs)

- 需求过程中确认的产品规则 (如「订单取消必须填原因」), 由 Agent **总结成 SPEC 草稿**留在 CHG 内; product/specs/ 是这些规则的「正式家」。
 - 晋升门槛就两条: ① **Agent 总结好草稿**; ② **人工评审通过**。评审通过即晋升进 specs, Skill 规则明确禁止 Agent 绕过评审直接写入。
 - **为什么**: specs 是后续所有需求的第二优先级输入源 (Approved Specs 仅排在 Approved Standards 之后), 一旦被 AI 自动写入, 污染的是整个项目的上下文。宁可空着, 不让 AI 免审晋升。
 - 所以「需求做完 specs 还是空的」是符合设计: SPEC 草稿还在 CHG 归档包里, 等人工评审通过后晋升。
-

六、难点与解决故事 (每个 1 分钟, 讲出「问题 → 方案 → 代价」)

6-1 目录改名漂移 (体现演进能力)

问题: 目录用业务名, 树上改名后旧目录就「找不到主人」了——纯文本目录没有身份。**方案 v1**: README front-matter 写 id 作锚点, 重跑时按锚点 rename。**代价**: 锚点读取、冲突检测、drift 扫描一堆逻辑, 且「README 已存在跳过」策略让陈旧描述长期驻留。**方案 v2 (方案 D)**: 重新审视后承认——目录本来就是派生物, 派生物就不该有身份。features/ 改为「先清后建」的全量重建, 锚点机制只在 CHG 内部 (真正的长期数据) 保留。**结论金句**: 「能重建的不要同步, 是派生的就不要给它状态」。

6-2 防止 Agent 绕流程 (体现安全设计)

问题: Agent 有完整的文件写权限, 理论上可以直接改 metadata 把状态推到 completed。**方案**: 纵深防御——① 所有状态写入入口到 Transition Service 单文件; ② gate 记录 artifact hash, 推进时 hash 不符就拒绝; ③ Machine Gate 只检查「明确声明为机检项」的条目, 避免粗暴扫全部 checkbox 被话术糊弄; ④ Human Gate bypass 强制留痕。**诚实补一句**: 这是「防呆不防坏」, 真要恶意改文件 Git 历史看得见——可审计性就是兜底。

6-3 多仓一致性 (体现分布式思维)

问题: 一个需求改 3 个仓, Workspace 怎么知道每个仓做到哪了? **方案**: DU 抽象 + commit 指针 + fan-in 机检。Workspace 不碰代码只记账; test 阶段完成的前提是所有 DU 的 evidence 齐了 (du-fan-in-testing)。dir 型仓库降级为目录存在性检查——尊重不同团队的工程现状。

6-4 文档即数据库的一致性 (体现细节)

问题: 状态散在 YAML, 改坏一个字段全盘瘫痪。**方案**: parseDocument 保留注释改写; patchStatus 前必过状态机 validateTransition; 核心状态机/转移服务/归档器有直接单测 (合法链/跳阶段/回退/hash stale/终态) 锁行为。

七、工程质量 (数字 + 做法)

- **379 个测试全绿**, node:test 原生跑, 无测试框架依赖; 测试覆盖: 状态机、Transition Service (含 hash stale)、归档、DU、Gate、Workflow 断点、输入校验、doctor 结构检查、骨架物化。

- **输入前置校验**: CHG id 强制 CHG-`\d{4}`、唯一性双侧查 (changes+archive)、DU id 正则前移到 IO 之前——脏数据挡在门口而不是 doctor 才发现。
- **-json 机器可读输出**: workflow/gate/du 全支持, Agent 集成从「解析屏幕文案」升级为「读协议」。
- **-dry-run**: upgrade 和 materialize 都能先看差异再执行。
- **CLI 薄壳**: 命令层只做解析/展示/退出码, 业务全在 core 纯函数——CLI 换壳 (比如未来出 web 视图) 不动内核。

八、高频追问 Q&A (尖锐问题预演)

Q1: 这跟直接让 Claude Code 开 plan mode 写个 CLAUDE.md 有什么区别?

CLAUDE.md 是「提示词」, 没有状态、没有校验、没有生命周期。我的框架里流程状态外置在磁盘、由状态机裁决、有双 Gate 和 hash 一致性校验、有审计包。提示词会漂移, 协议不会——skill 和命令是程序化渲染的, 方法论只有一份。

Q2: 为什么是 CLI + 文件, 不上数据库/服务端?

目标用户是个人开发者和小团队, 流程资产要进 Git、要能离线、要能 diff。YAML+Markdown 让每一条历史都是人类可读的 diff; Git 本身就是版本化数据库。引入服务端反而把「谁都能跑」变成「谁都要部署」。

Q3: AI 执行产物质量不行怎么办?

分层兜底: Machine Gate 拦格式/完整性 (确定性), Human Gate 拦业务判断, Evidence 拦「有没有验证过」, converge 阶段知识回流后下一次 explore 的上下文质量更高。流程不保证产物对, 但保证**错的产物走不到下一步**。

Q4: 流程这么重, 小需求怎么办?

承认这是 trade-off, 也是演进方向: 已规划需求分级 (minor 走轻量通道: explore+prd 合并、design 可选、converge 简化)。当前版本先保证「重流程的每一步都有明确对手方」, 分级是产品化阶段的排序问题。

Q5: 为什么不让框架调模型, 端到端自动化?

刻意不做。模型调用引入不确定性、成本和厂商耦合; 把「执行」留给最擅长执行的 Agent, 把「流程裁决」留在确定性框架里, 两边各自演进。以后要自动化, 加一个「Agent 适配器」即可, 架构不用动。

Q6: 并发问题? 两个 Agent 同时跑一个 CHG?

当前版本是单写者假设 (个人/单人一个 CHG), 靠约定不靠锁。团队版 (Phase 4) 规划了 Approval 和权限, 届时在 Transition Service 收口处加文件锁/乐观锁即可——正因为所有写都过单点, 加锁是一处改动。

Q7: 测试怎么做的? 为什么不用 Jest?

node:test 原生运行器, 零依赖和项目哲学一致。核心策略: 领域层纯函数直接单测 + 临时目录做真实文件 IO 的集成测试 (创建/改名/归档都是真文件系统行为)。379 个用例, 重构目录体系时这套测试是底气。

Q8: 最大的收获/如果重做会改什么?

收获：AI 时代的工程化瓶颈不在生成代码，在**流程可信**。如果重做：① 更早引入「派生缓存」思维，少走锚点同步弯路；② 需求分级（轻重通道）应该在 v1 就设计进去而不是事后补。

Q9：知识沉淀怎么防止 AI 把错误经验写进知识库？

分层：技术规则走 converge 阶段回流 standards/，有机检和人审；产品规则更严格——晋升门槛就两条：**Agent 总结好草稿 + 人工评审通过**。Agent 只能把 SPEC 草稿留在 CHG 里，禁止直接写 product/specs/。specs 是后续所有需求的第二优先级输入源，一旦被自动写入污染的就是整个项目的上下文。所以你会看到 specs 可能暂时是空的——那是草稿还在归档包里等人工评审，不是知识没沉淀。

九、数字卡片（考前最后过一遍）

项	值
SDD Skills	11 个 (8 阶段 + 3 工具: feature-tree/knowledge/reverse)
测试	379 全绿
运行时依赖	4 个 (commander / @clack/prompts / picocolors / yaml)
IDE 集成	3 个目标 (Trae / Cursor / Claude Code), 11 条斜杠命令
Feature Tree	4 级 (L1 Module / L2 Feature / L3 Capability / L4 Story)
Change 生命周期	9 态
Workflow	5 态 (ADVANCED + 3 WAITING + COMPLETED)
返回	
四个世界	Standards / Product / Delivery / Implementation
prompts	5 族 (common/explore/design/coding/review)
分族	
upgrade	4 步固定 + rollback
命令族	change / du / workflow / gate / skill / feature / doctor / init / upgrade / ide / status / validate / context

十、开场白模板（串场用）

「这个项目源于一个观察：AI 编码工具解决了『写得快』，没解决『管得住』。我的角色定位很清楚——不做模型层，做工程控制层。整个框架围绕一条主线：**把软件工程里的流程纪律，翻译成 AI 能执行、框架能裁决、人能审计的协议**。接下来我挑三个最有代表性的设计展开讲：状态推进单入口、双 Gate 分工、和多仓交付。」

(然后按五-1、五-2、五-7 讲，被追问再展开其他节。)

备注：本文件是个人面试材料，不属于项目交付物，可随时删除。